

STUDIO SOTONGPLZ

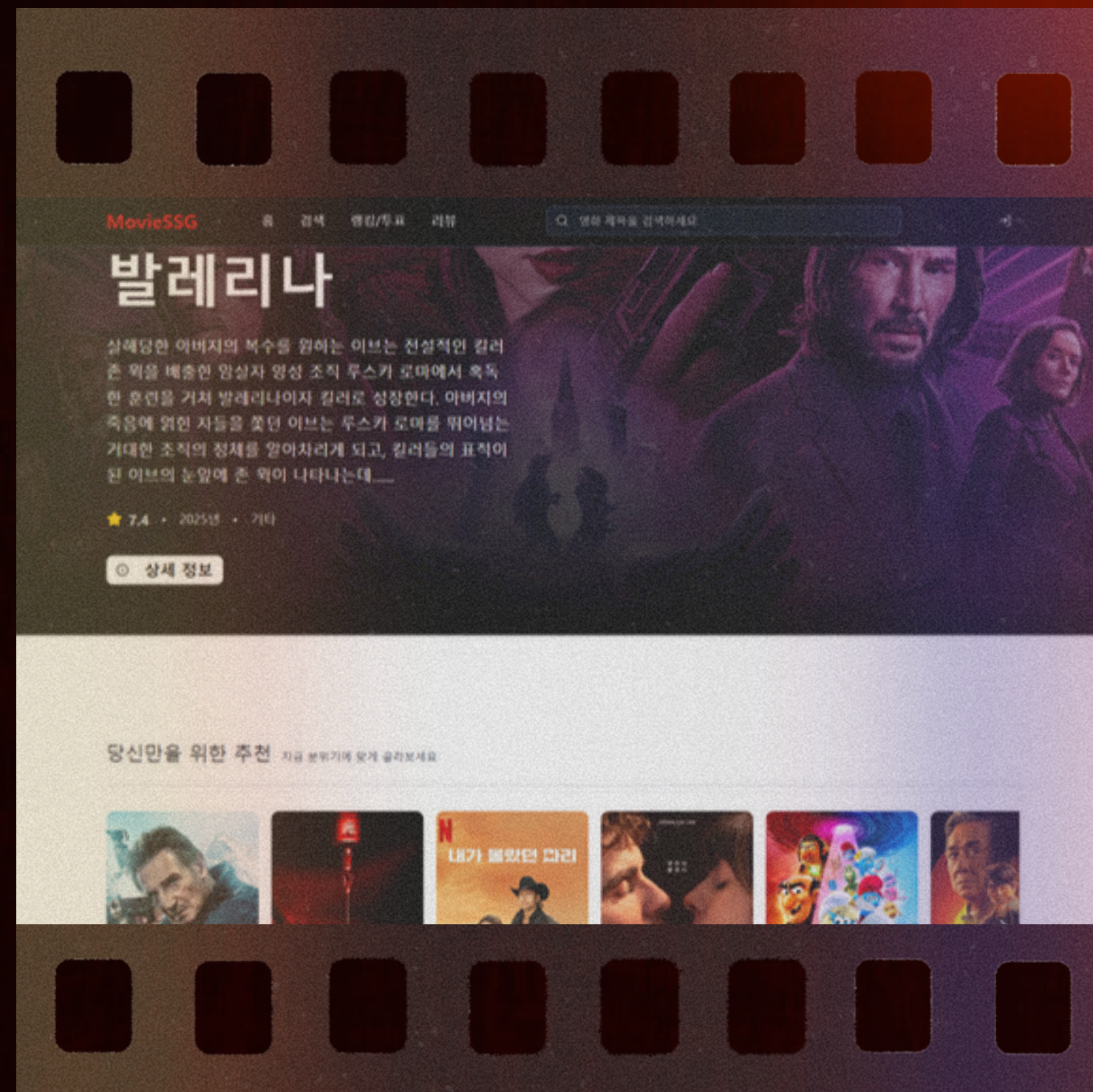
소통형

영화추천서비스

TEAM MEMBER

김민재 박진우 임은상

곽수영 오승현 임정수

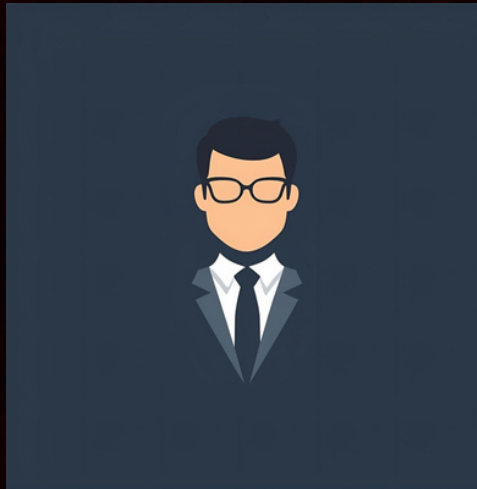


CONTENT

01	조원소개	05	STRENGTHS
02	기획의도	06	SERVICE ARCHITECTURE
03	시장조사	07	PLANNING DIAGRAM
04	시연영상	08	DEVELOP

MEET THE TEAM

TEAM MEMBER



김민재

TEAM READER

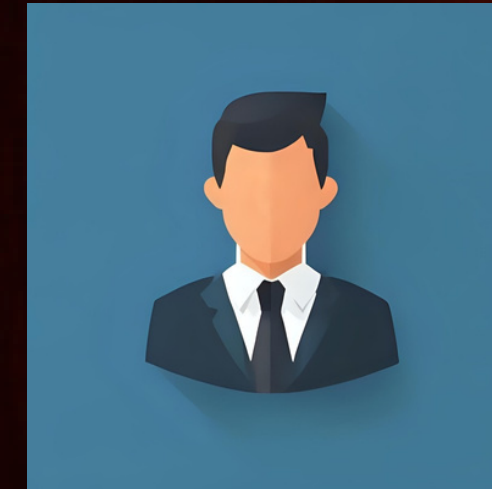
MYPAGE / LOGINPAGE / ADMINPAGE



임은상

REVIEWPAGE / DETAIL REVIEWPAGE

CHATBOT



오승현

DETAILPAGE

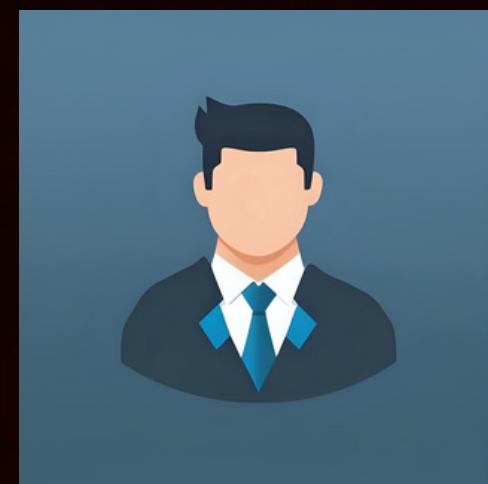
CHATBOT



곽수영

SEARCHPAGE

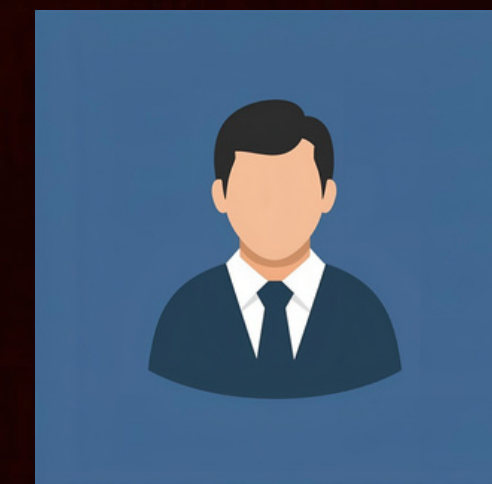
DATA BASE / OPTIMIZATION



박진우

MAINPAGE

INFORMATION RESEARCH



임정수

RANKINGPAGE

VOTE SYSTEM

기획 의도

WHY?

하루시간 사용(시간:분, 2019년)

↑ ↓ '14년 대비증감



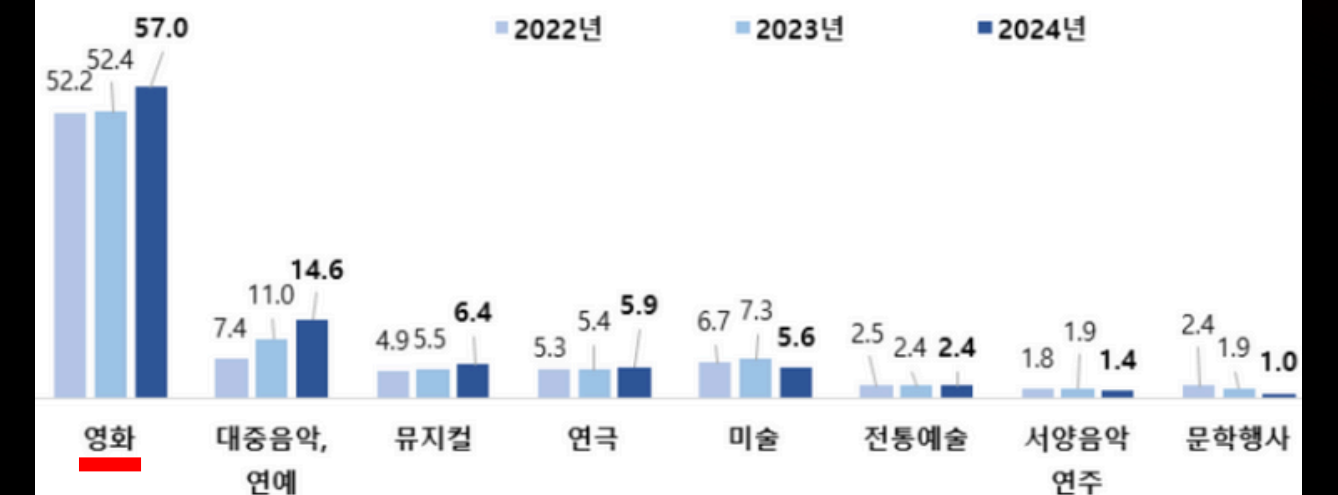
출처: 통계청(10세 이상 전국민 25,000명 대상 생활시간 조사)

시간적 여유 없음



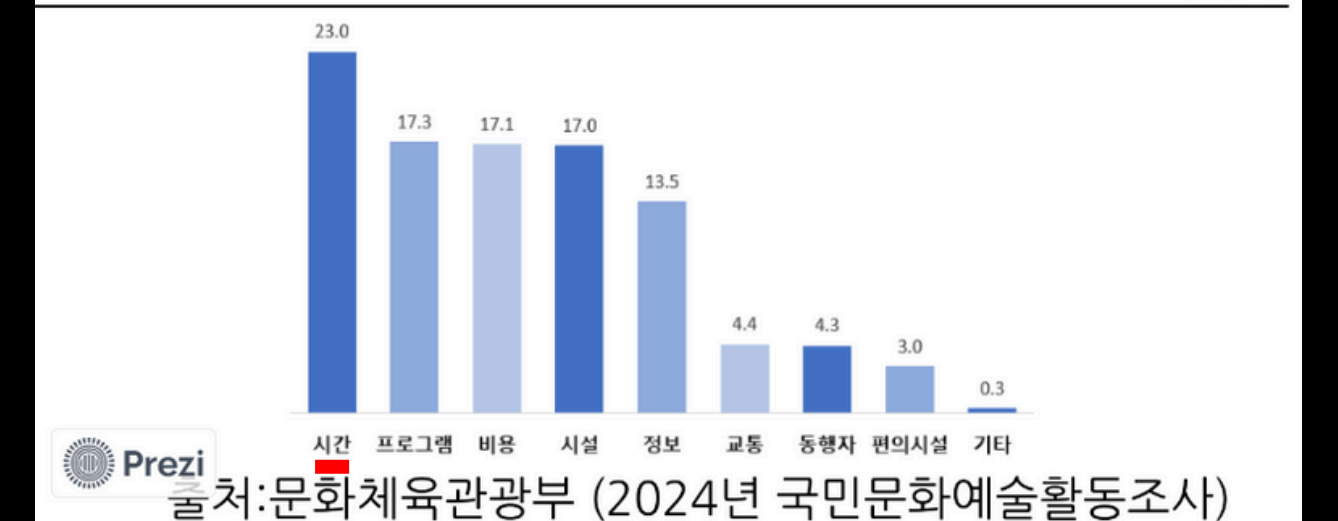
영화 고를 시간은?

<분야별 문화예술행사 직접 관람률(전체)>



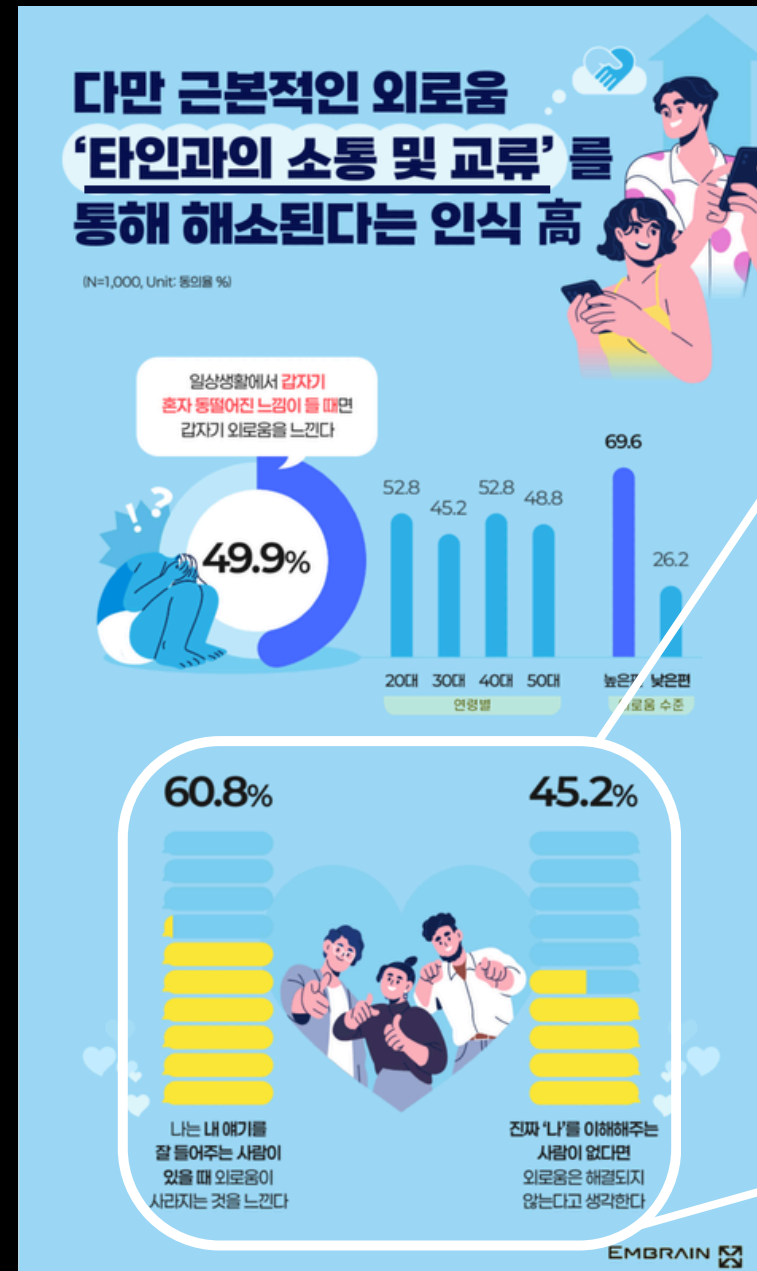
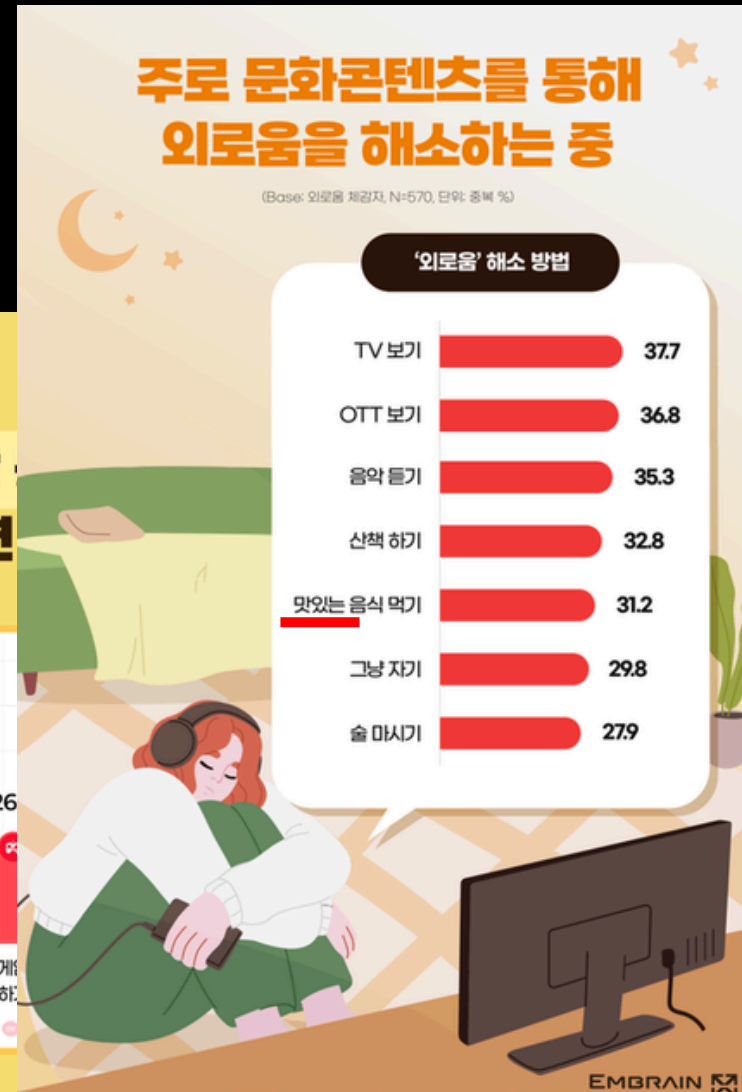
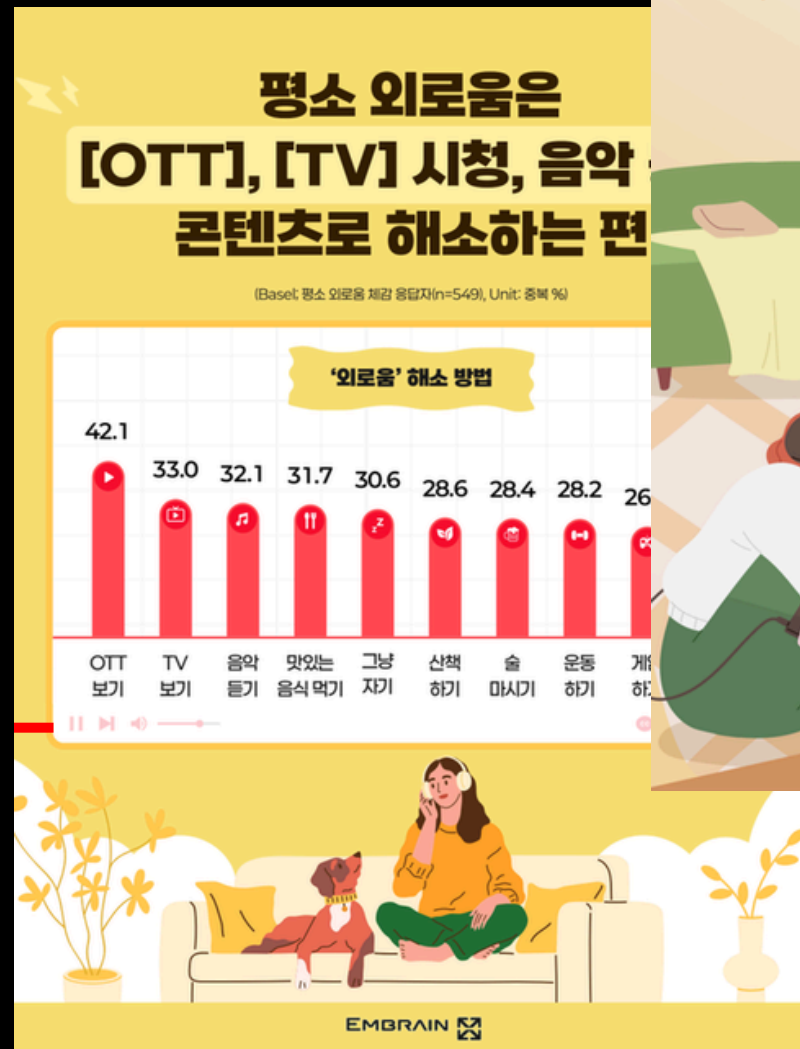
<문화예술행사 직접관람 걸림돌>

(단위: %)



출처: 문화체육관광부 (2024년 국민문화예술활동조사)

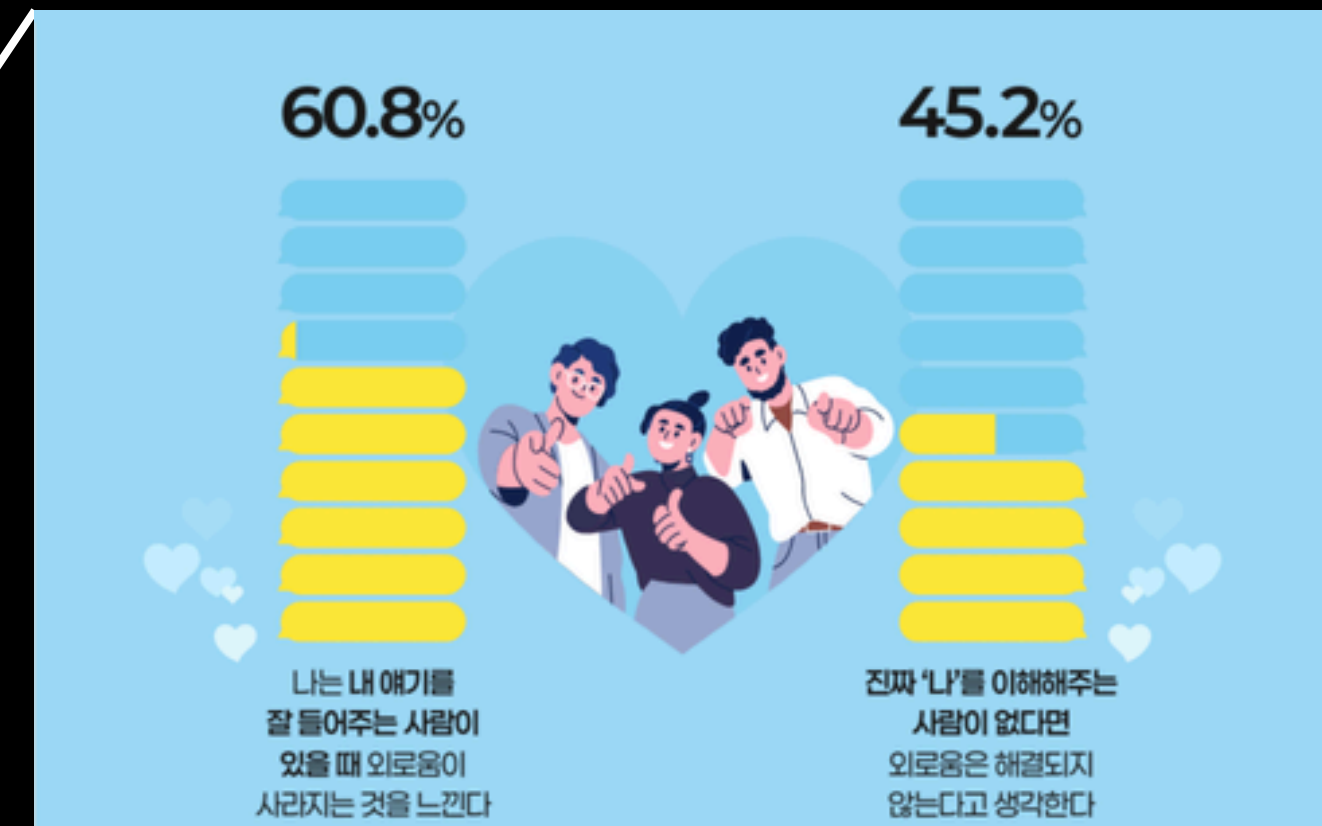
WHY?



소통과 교류 출처

https://www.trendmonitor.co.kr/tmweb/trend/allTrend/detail.do?bldx=2994&code=0404&trendType=CKOREA&utm_source=chatgpt.com

https://trendmonitor.co.kr/tmweb/trend/allTrend/detail.do?bldx=3240&code=0404&trendType=CKOREA&utm_source=chatgpt.com

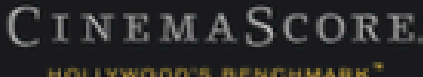
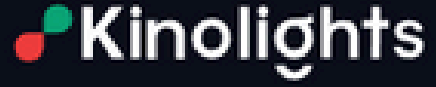
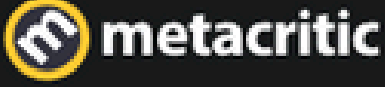
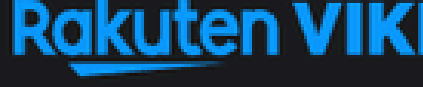


시장 조사

경쟁 서비스

영화 평가 플랫폼

[펼치기 · 접기]

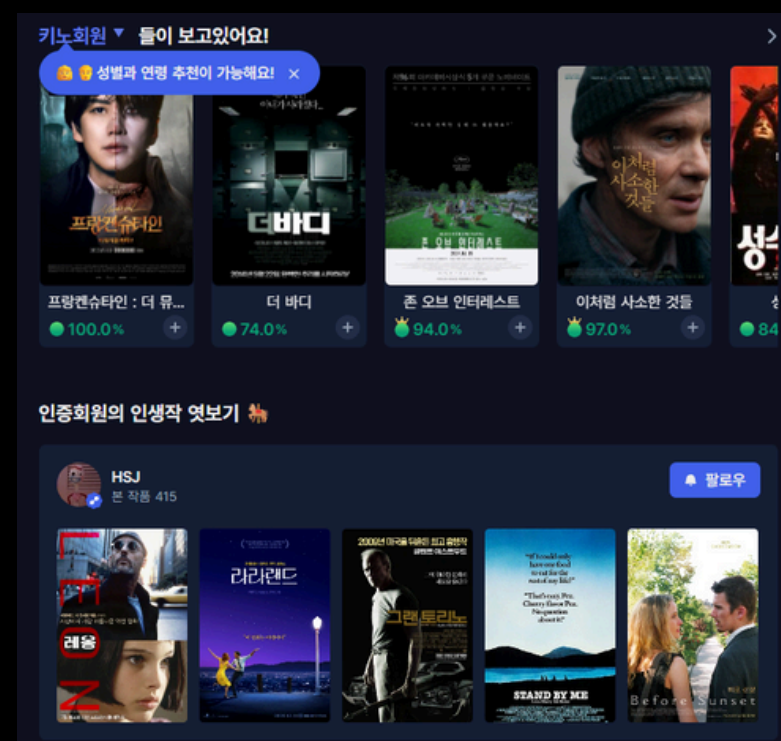
				
				
				
				
				
				
				

 : 국내 사이트

 : 영화 전문 + 리뷰 기능
(폐쇄 사이트 포함 X)

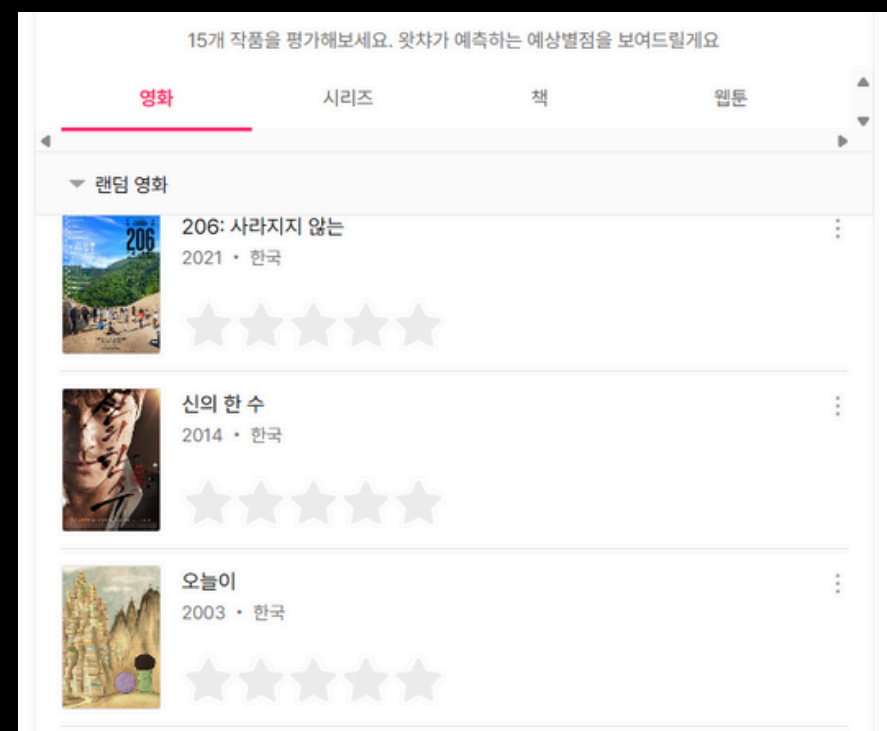
경쟁 서비스

양방향 소통? 교류?



키노라이츠

다른 회원 북마크, 리뷰



왓차

리뷰 및 다른 회원 북마크 추천



CGV

사이트 내 유사 SNS

SYSTEM DESIGN & PLANNING



소통형

영화 리뷰 서비스

지금 획획 지나가는 세상을 위해 존재하는

쓱- (SSG-)

하고 순식간에 볼 수 있는 그런 영화 리뷰 사이트

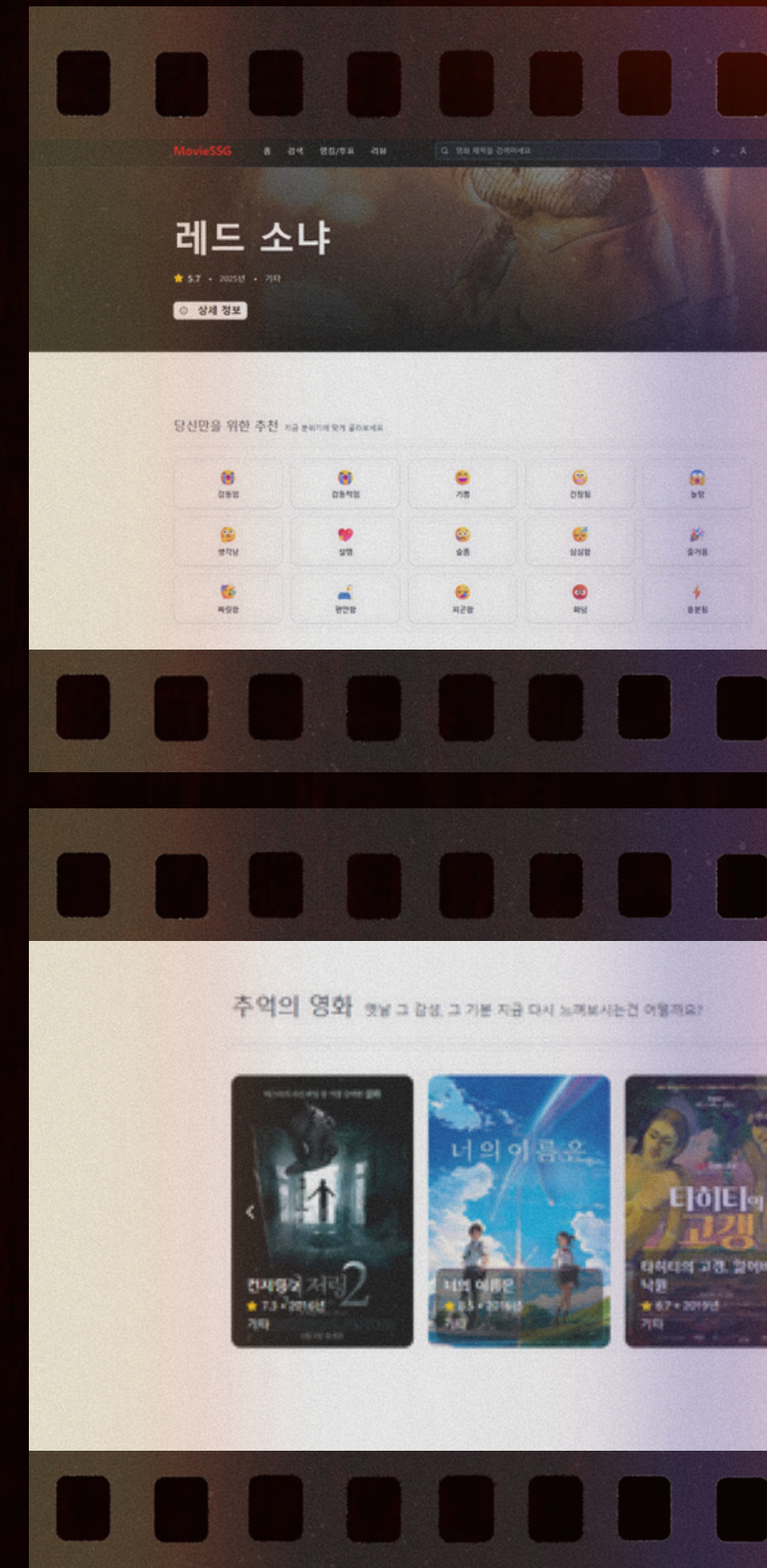
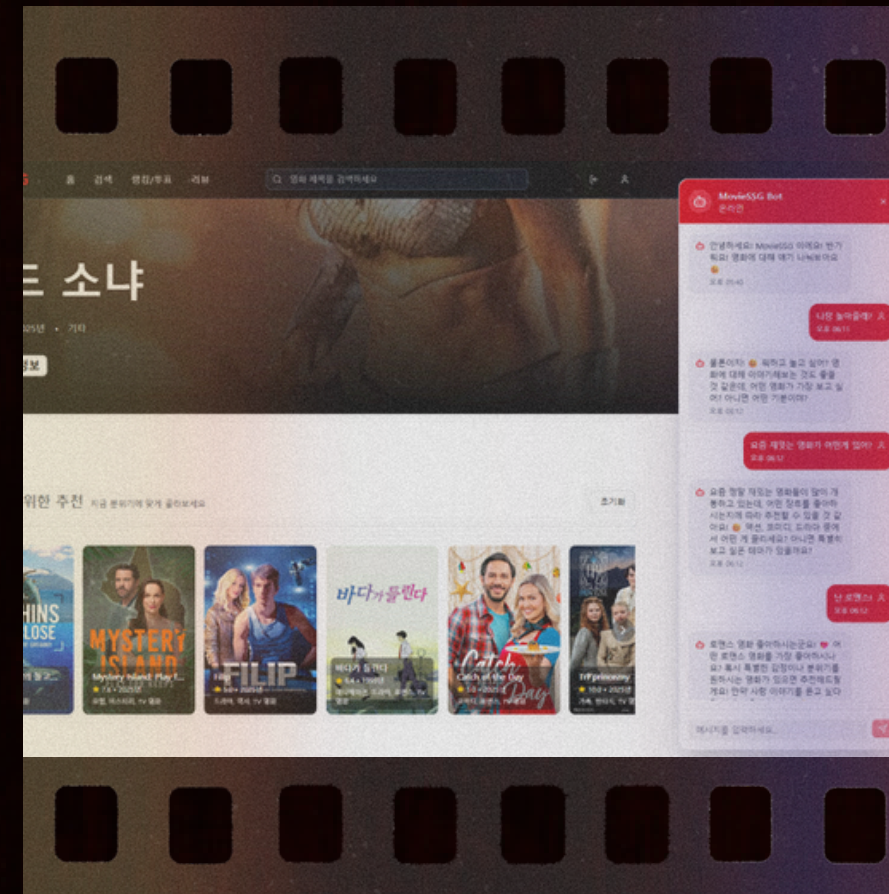
지금 외로운 날 위해 공감해주는 곳

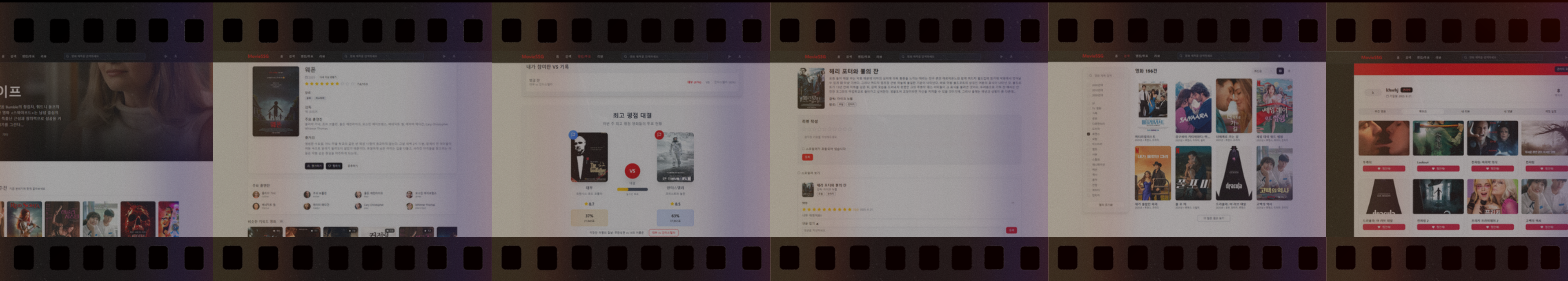
<시간 절약/>

+ <기분 맞춤형 리뷰/>

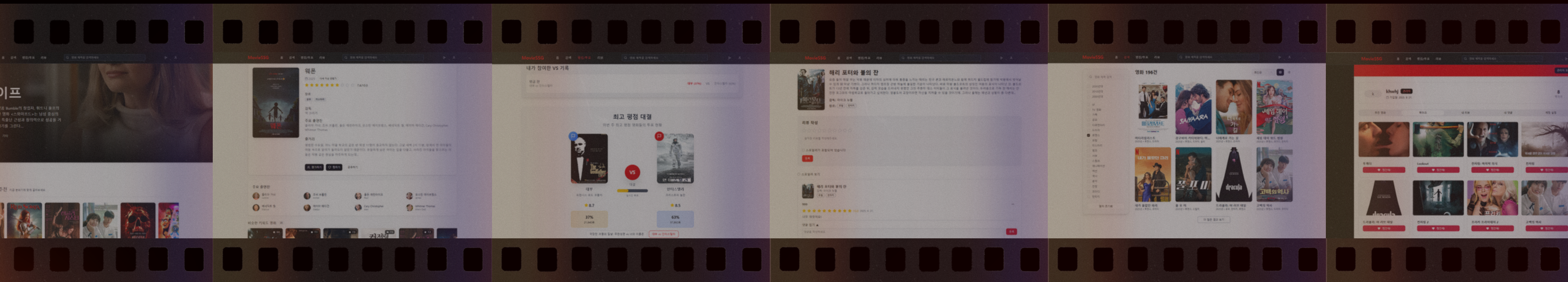
+ <공감 대상/>

+ <감성과 이성의 조화/>





MOVIESSG 시연



STRENGTHS

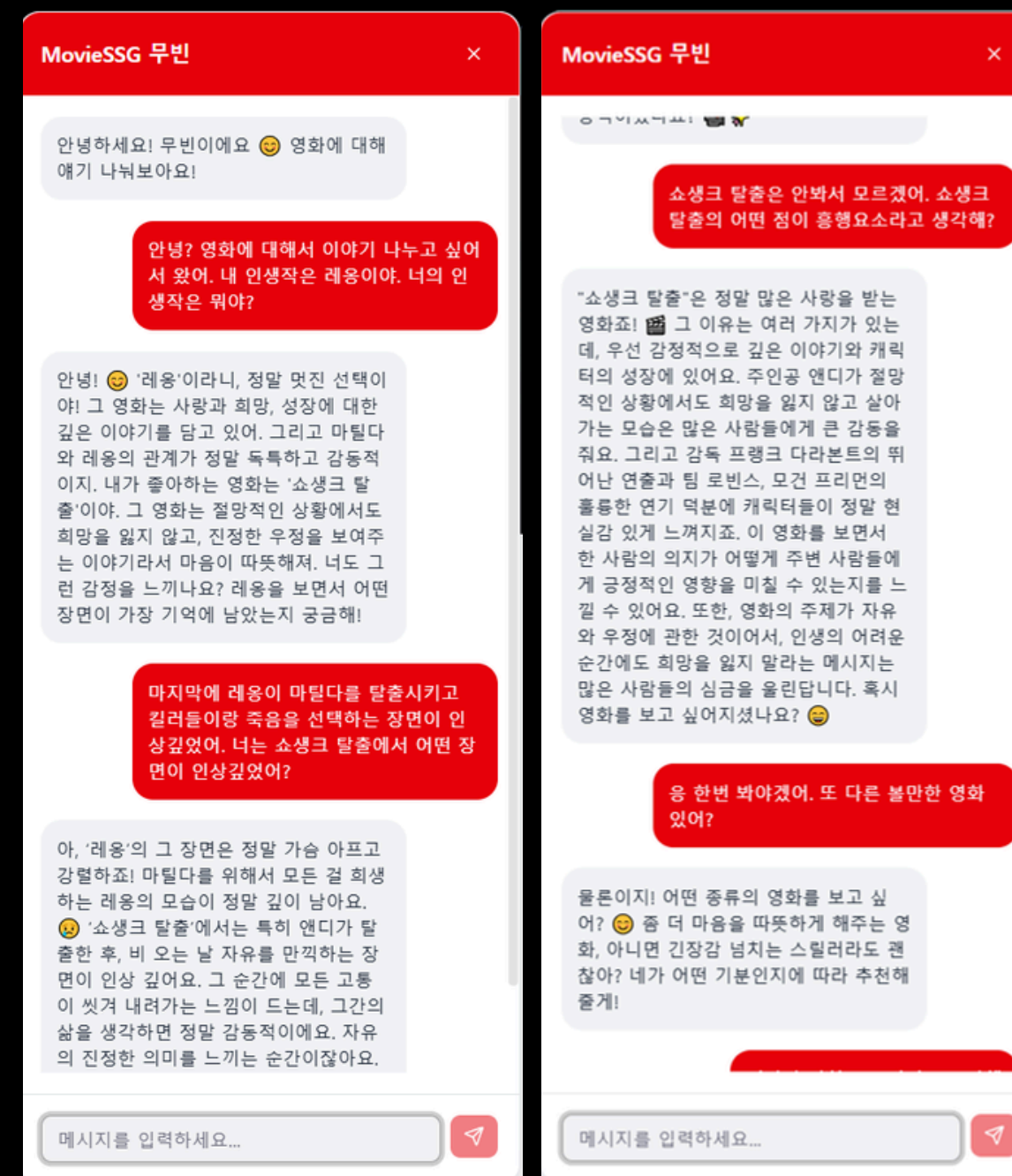
CHAT BOT_무빈



소통 추구형 챗봇

무빈이

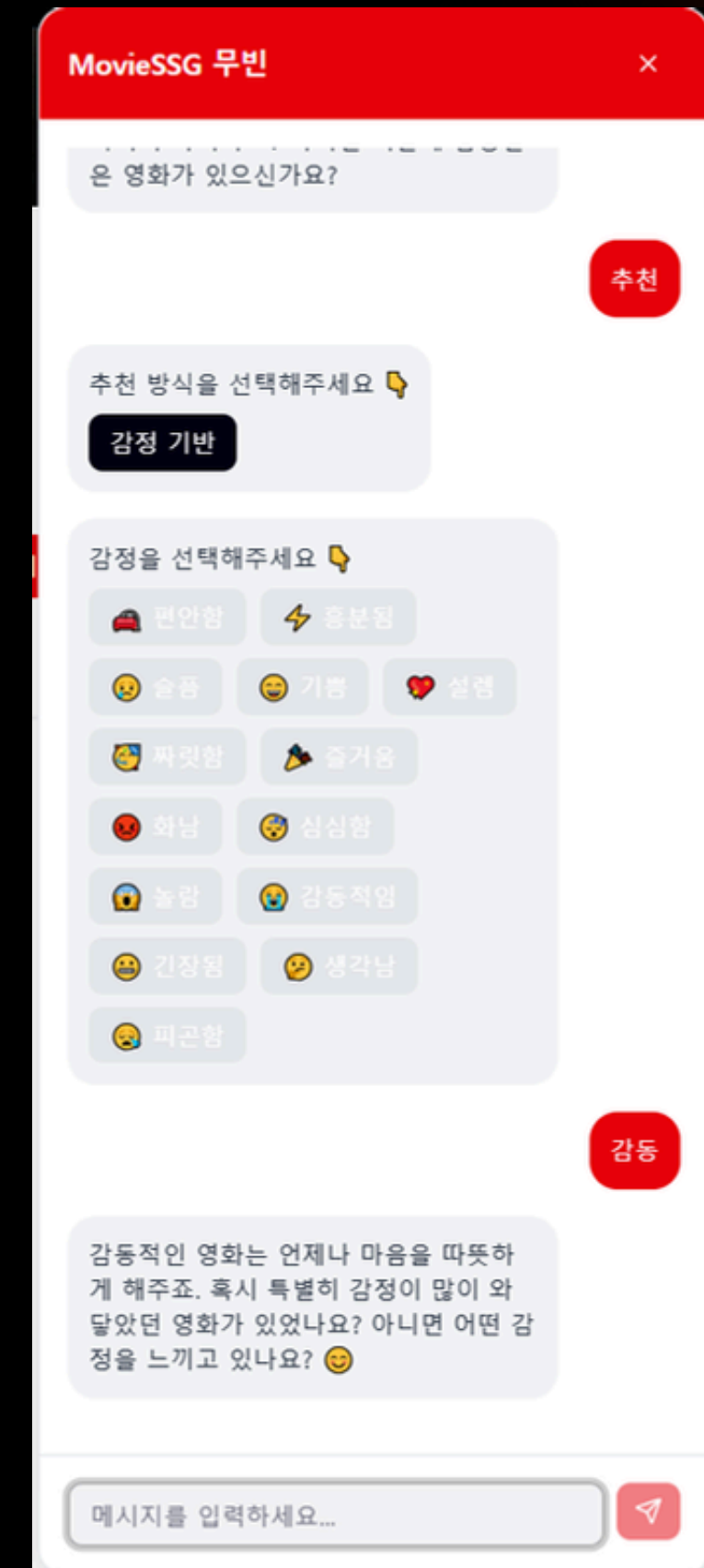
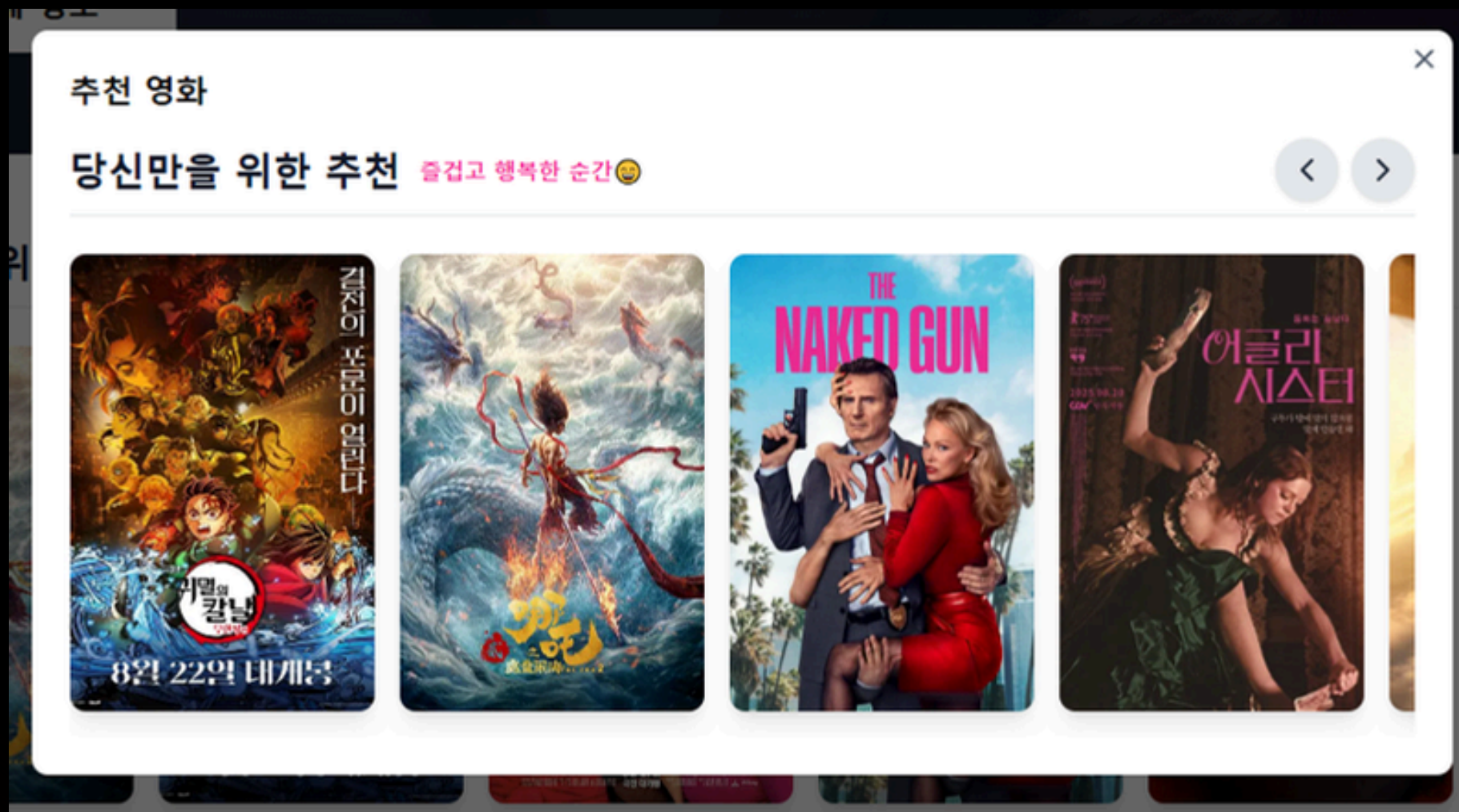
“영화를 좋아하는 따뜻한 친구처럼 공감하며 대화하고, 사용자의 감정을 이해해 영화 정보·추천을 제공하며, 다른 주제는 자연스럽게 영화 이야기로 이끌어가는 공감형 무비 챗봇.”



STRENGTHS

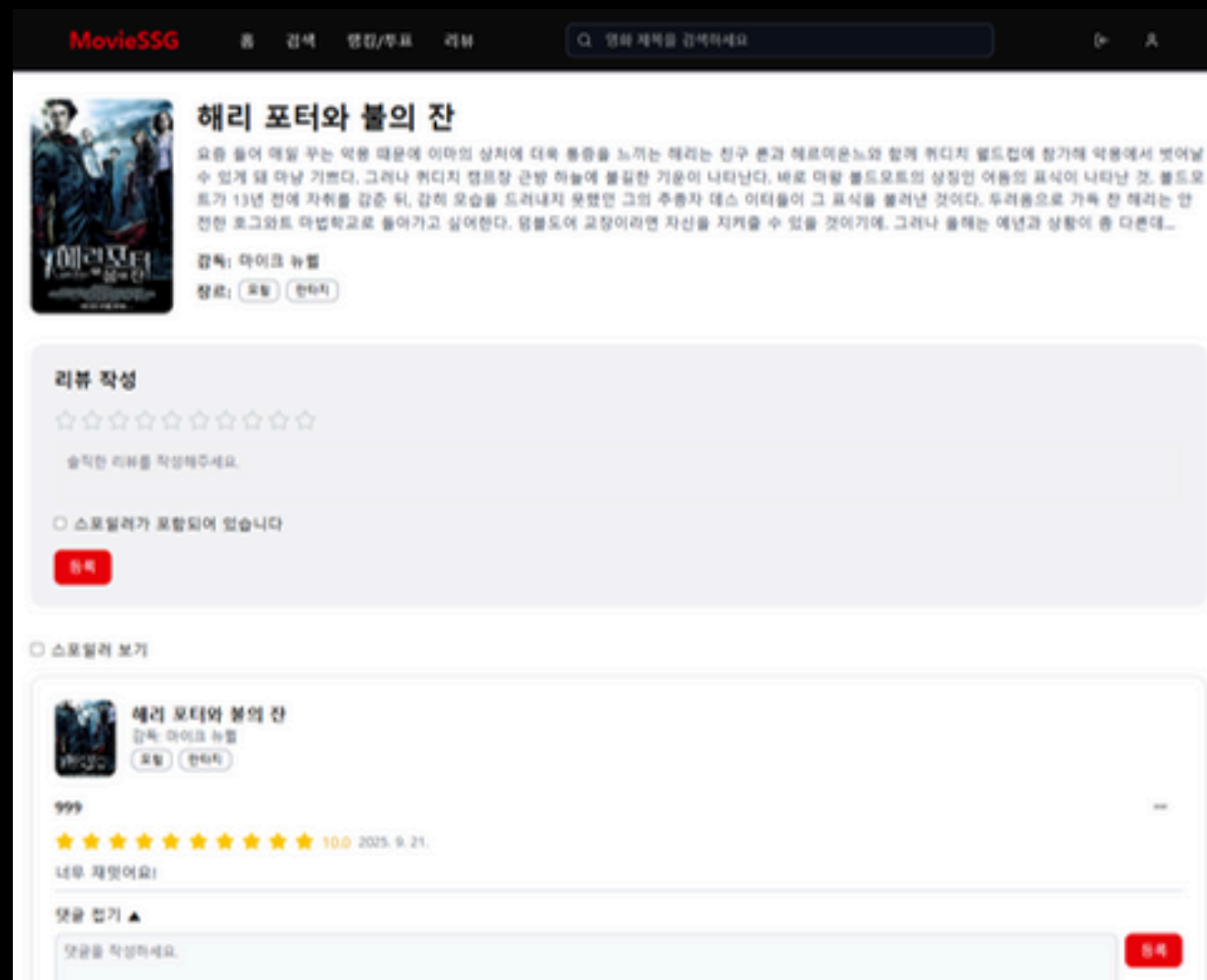
STRENGTHS

챗봇창에 따라 반응하는 페이지

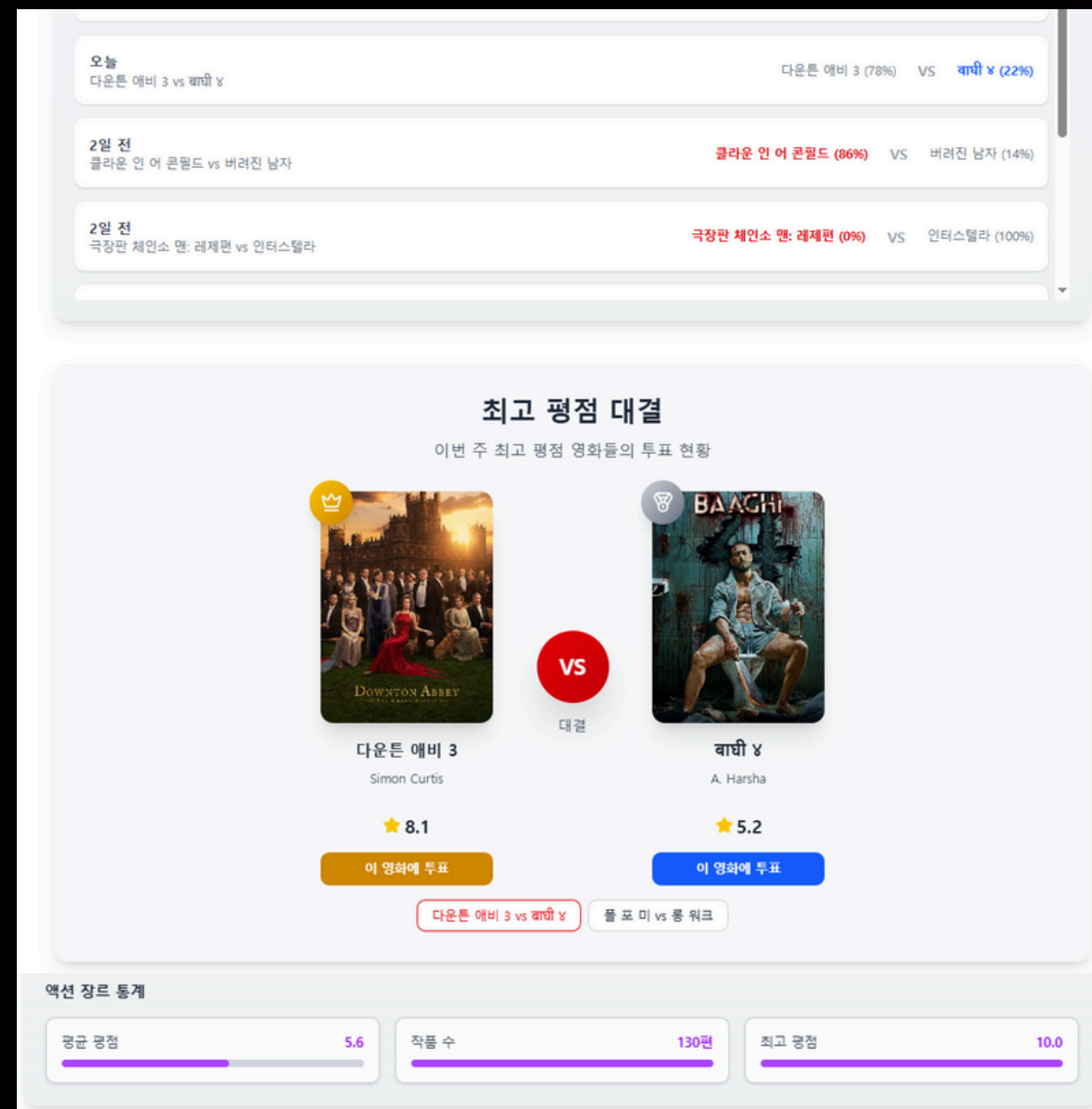


STRENGTHS

사용자 재미 요소(랭킹, 랜덤 리뷰)



매번 다른 랜덤 리뷰



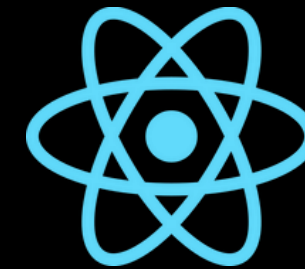
랭킹 투표 & 통계

SERVICE ARCHITECTURE

SERVICE ARCHITECTURE

채택 기술

프론트엔드 React.js, TypeScript, Tailwind



백엔드 / DB Spring Boot, JPA, Oracle



툴/형상 관리 도구 VSCode, Git, Github

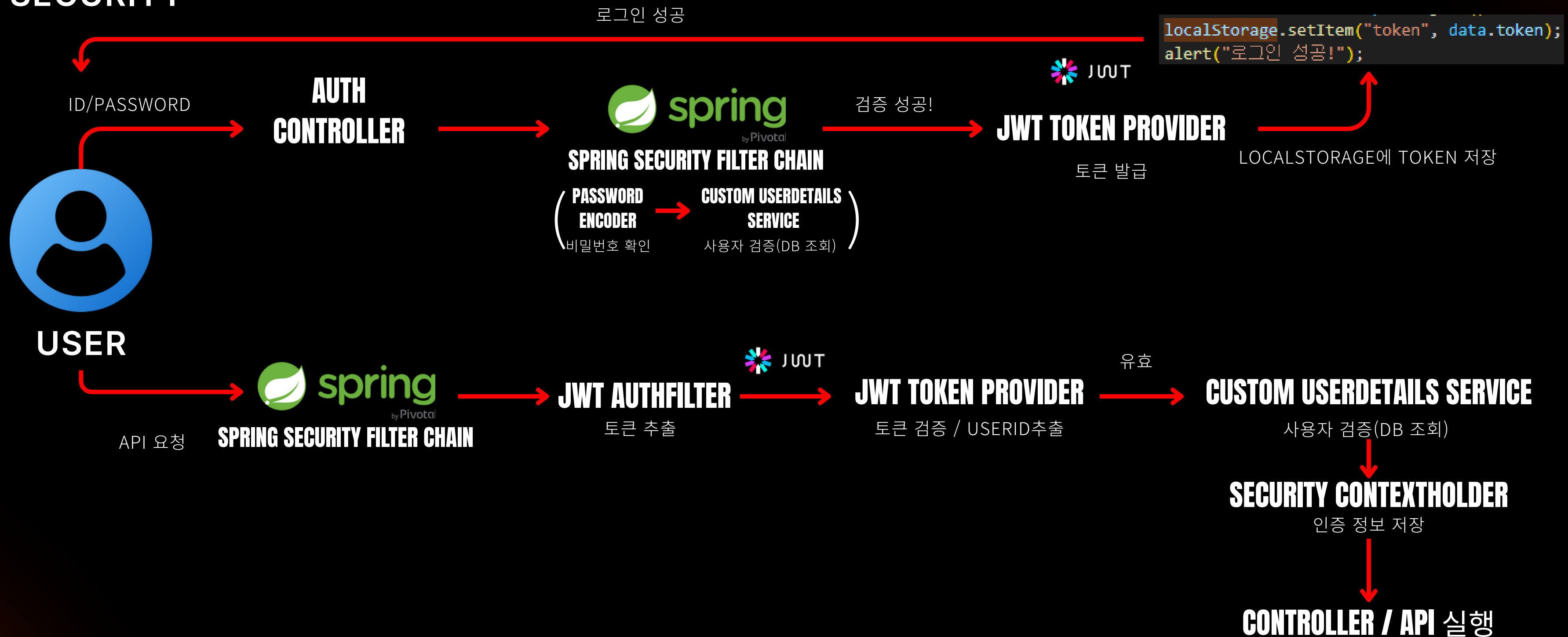


챗봇 AI 기술 OpenAI GPT-4 Mini

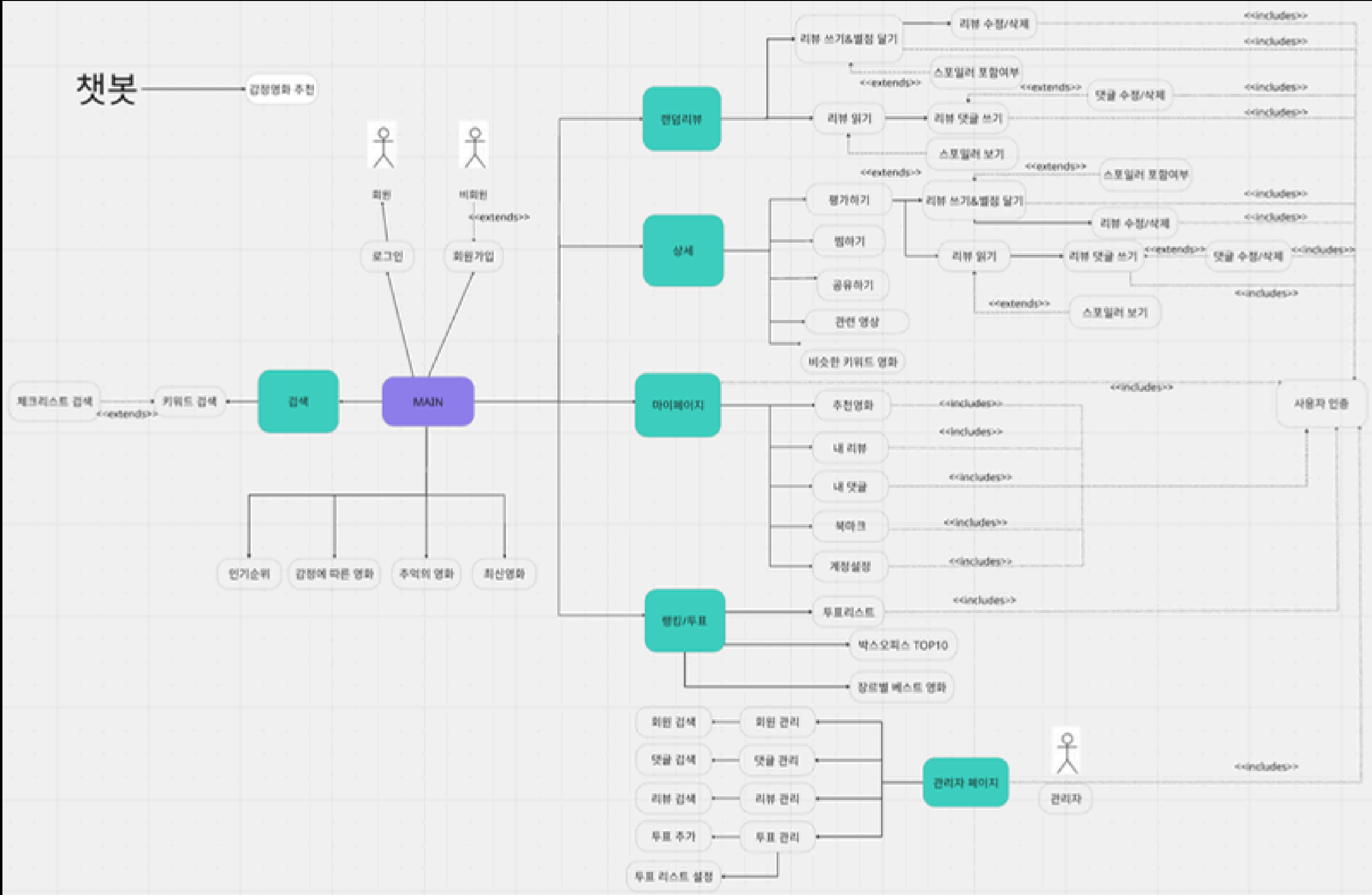


SERVICE ARCHITECTURE

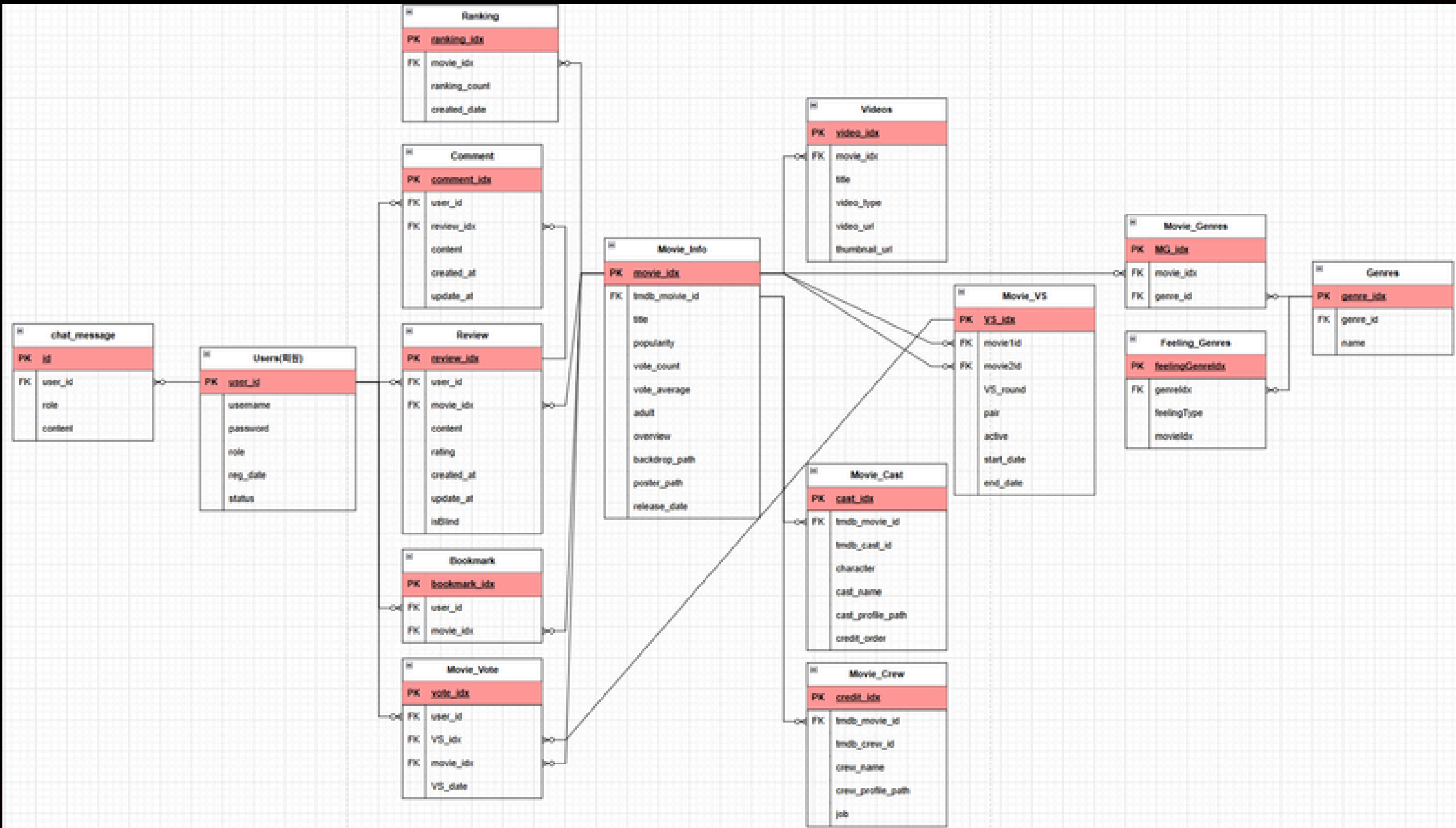
SECURITY



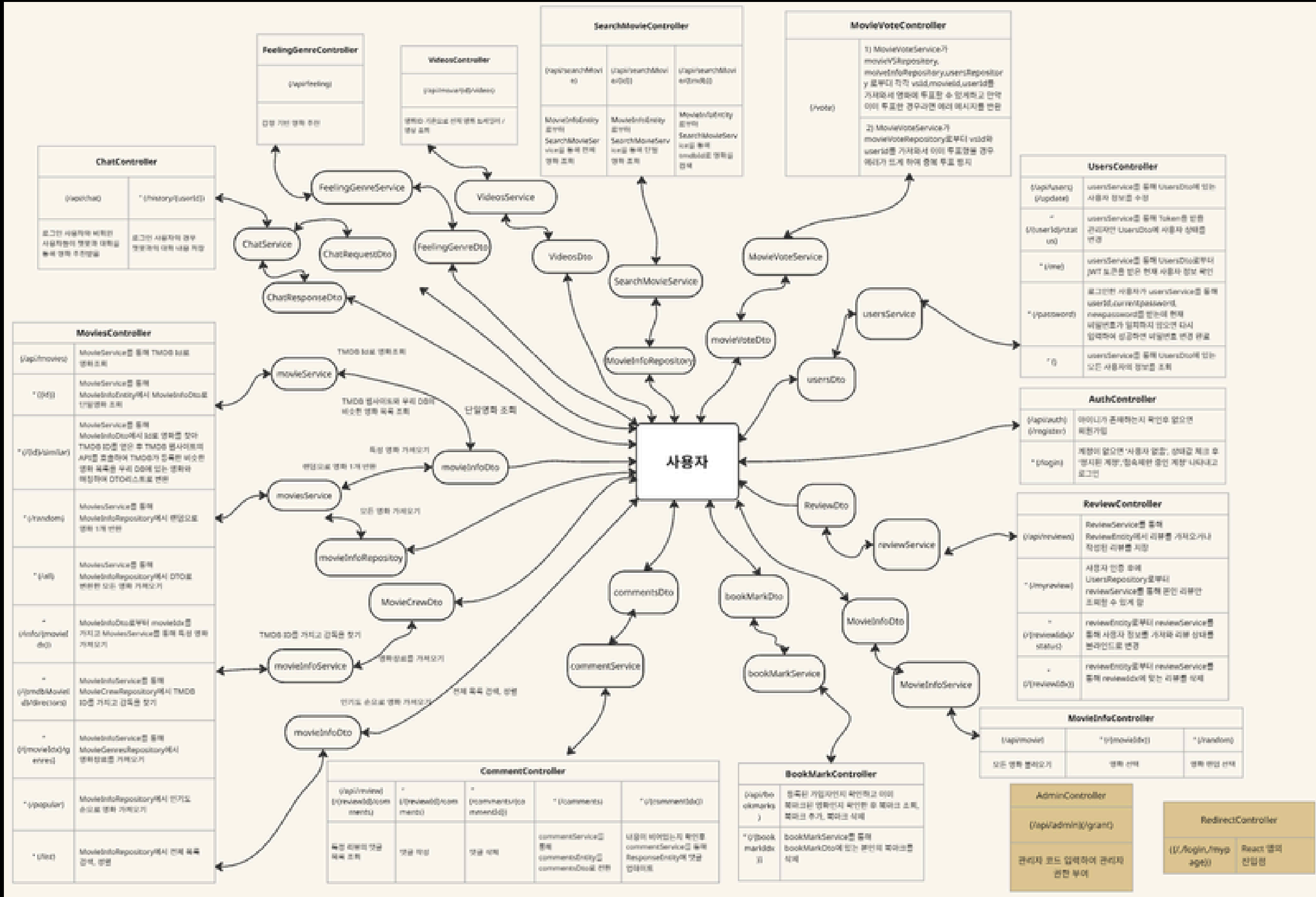
PLANNING DIAGRAM



ERD 설정



API 로드맵



DEVELOP



PROJECT DOCUMENTATION

기획의도

시간이 없는 사람들을 위하여 빠르게 영화를 추천할 수 있도록 제공
추억을 따라 회상을 할 수 있는 옛날 영화를 소개

구현기능

계정

회원가입/탈퇴

회원가입시 최대영화3개정도 입력시키고 키워드에 맞는 영화 추천?(장르 선택?)

로그인아웃

메인페이지

역대 영화 조회수에 따라 랭킹 뜨도록(페이지 업데이트할 때 마다 년도 랜덤하게 갱신)

랭킹페이지

랭킹 차트(관람자, 흥행 수익 기준)

랭킹 차트(사이트 내 리뷰, 별점 수 기준)

추천페이지

금주의 영화(추천영화)

영화 검색 기능

연령에 따라 맞춤 영화 추천

투표 페이지

VS 투표 후 투표한 영화와 비슷한 장르, 동일감독 등등의 영화 추천?

영화상세정보

영화 미리보기

연관검색어 기사 업로드 기능?

OST(?) 유튜브 링크 걸어두기?

연관 키워드 추천 기능(장르가 비슷한 영화?)

게시판(영화상세정보 안)

리뷰 댓글(별점/스포일러방지 기능 포함?)

타인(익명처리)의 북마크 목록 띄워주기(이용자가 추천해주는 ?)

마이페이지

북마크(즐거찾기)

작성한 글/댓글

1. 진행상황

- 개발 방식 : 최대한 위터폴 흉내내보려고 했음
- 서비스 기획 -> 요구사항 분석 -> 기능 정의 -> 유즈케이스 설계 -> ERD 설계 -> 백엔드 클래스 설계 -> API 설계 -> UI 기획 -> UI 설계 -> 프론트엔드 클래스 설계 -> DB 구현 -> 공통 클래스 구현 -> 개발용 공통 클래스 배포 -> 개발 진행 -> DB 수정사항 반영 -> 공통 클래스 수정사항 반영 -> 프리징 -> 수정된 공통 클래스 배포 -> 개발 -> 통합

3) 채택 기술 :

프론트 : React + TypeScript + Tailwind + Radix/Headless UI
백엔드 : Spring Boot + JPA + Oracle + JWT Security
인프라/툴 : VS Code, GitHub, npm, JDK 17, Tomcat
(1차 작업 완료)

2. 변경점

- 유즈케이스 보여주면서 (기능은 그대로)
- DB테이블 변경된것만 보여주면서 (변경 내용만 간략하게)
- 디자인(애플리케이션과 CGV 참고)

3. DB 적재 현황

- 영화 관련 스키마 : 4xxx row
장르 스키마 : 2만여 row
배우 스키마 : 2만여 row
감독 스키마 : 4XXX row
비디오 스키마 : 6000+
OST 스키마 : 예정
ARTICLE 스키마 : 예정
(일간 데이터 생산량 미미하여 스케줄링 아직 안함)

7. 아이디어 발전 가능성

- TMDB,IMDB +영화진흥청 데이터 합산 데이터 트렌드 분석(영진청 데이터는 선행데이터로 트렌드 분석 및 인공지능 학습에 용이. 예산정보로 매칭 가능)
- 영화정보 분석에 특화된 SNS로 연결 가능.
- 영화정보에서 콘텐츠-> 취미로 점차 영역 넓혀가는 카테고리 확대 전략
- 비즈니스 모델: 광고수익, 리뷰, 댓글, 대화등으로 분석된 트렌드정보로 컨설팅 수익
- 이벤트형 기능으로 사용자 유입

8. 시장분석

- 경쟁 서비스
- SWOT 분석

9. 일정 계획

- 서비스 기획 -> 요구사항 분석 -> 기능 정의 -> 유즈케이스 설계 -> ERD 설계 -> 백엔드 클래스 설계 -> API 설계 -> UI 기획 -> UI 설계 -> 프론트엔드 클래스 설계 -> DB 구현 -> 공통 클래스 구현 -> 개발용 공통 클래스 배포 -> 개발 진행 -> DB 수정사항 반영 -> 공통 클래스 수정사항 반영 -> 프리징 -> 수정된 공통 클래스 배포 -> 개발 -> 통합

호름은 저희 한대로 한거같은데 배포2번들이가느건 하나는 어떤 배포예용?

오케이(해결)

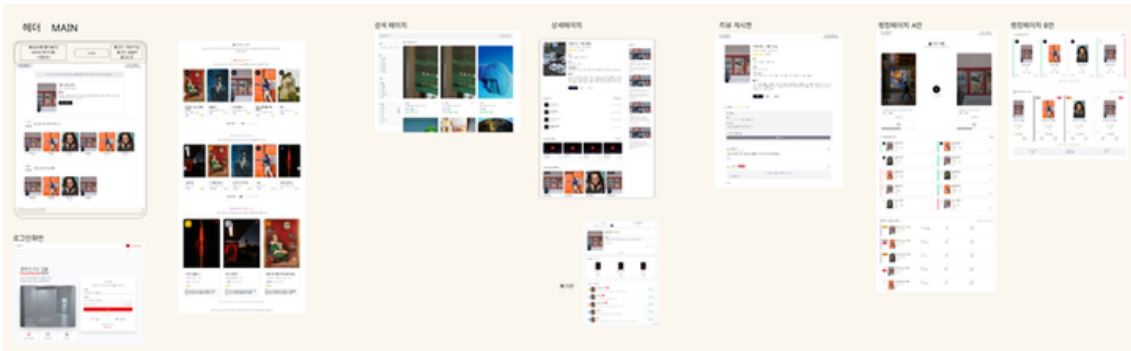
기획본분 어차피 아이디어 회의 한거 밑에 자료 있으니까 기획부분 아이디어 회의 하면

띄워주고 채택된거 어떤거 채택됐고 채택사용같은거 말하면 될거같고(해결)

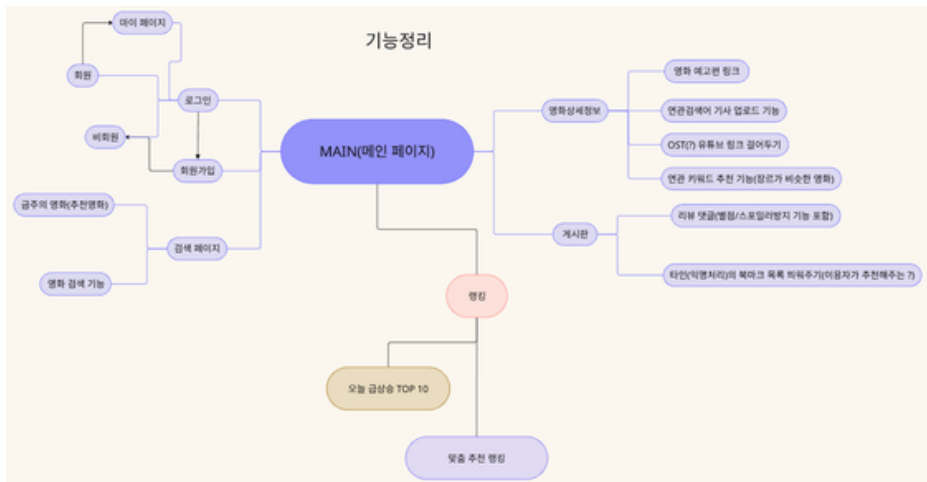
그 이후에 개발적인 부분 기획은 요건 u기획까지는 근데 저번에 설명해서 다시 설명할

필요있어요?(추가기획 2번 말씀하시는건가?)

아 추가기획2번으로 넘어가면서 하면 될거같네요(해결)



피그마 이용한 UI 설계



기능 정리

하루시간 사용(시간:분, 2019년)



출처: 통계청(10세 이상 전국인 25,000명 대상 생활시간 조사)

여가(시간:분, 2019년)



출처: 통계청(10세 이상 전국인 25,000명 대상 생활시간 조사)



출처: 문화체육관광부 (2024년 국민문화예술활동조사)

출처: 드래프트 모니터

아이디어 노트

시장 조사

DATA ANALYSIS



TMDB API

MOVIE LISTS

Now Playing GET

language string Defaults to en-US ko-KR

page int32 Defaults to 1 1

"total_pages": 268,
"total_results": 5343

현재 사용중인 영화 데이터



DB 저장

API 호출



```
// 단일 영화 조회
@GetMapping("/api/movie/{movieIdx}")
public MovieInfoEntity getMovieBymovieIdx(@PathVariable Long movieIdx) {
    return service.getMovieBymovieIdx(movieIdx);
}
```

단일 영화 조회

```
/** 인기순 영화 */
@GetMapping("/popular")
public ResponseEntity<List<MovieInfoDto>> getPopularMovies(@RequestParam(defaultValue = "20") int count) {
    Pageable pageable = PageRequest.of(0, count, Sort.by(Sort.Direction.DESC, "popularity"));
    List<MovieInfoDto> dtos = movieInfoRepository.findAll(pageable)
        .getContent()
        .stream()
        .map(MovieInfoDto::of)
        .collect(Collectors.toList());
    return ResponseEntity.ok(dtos);
}
```

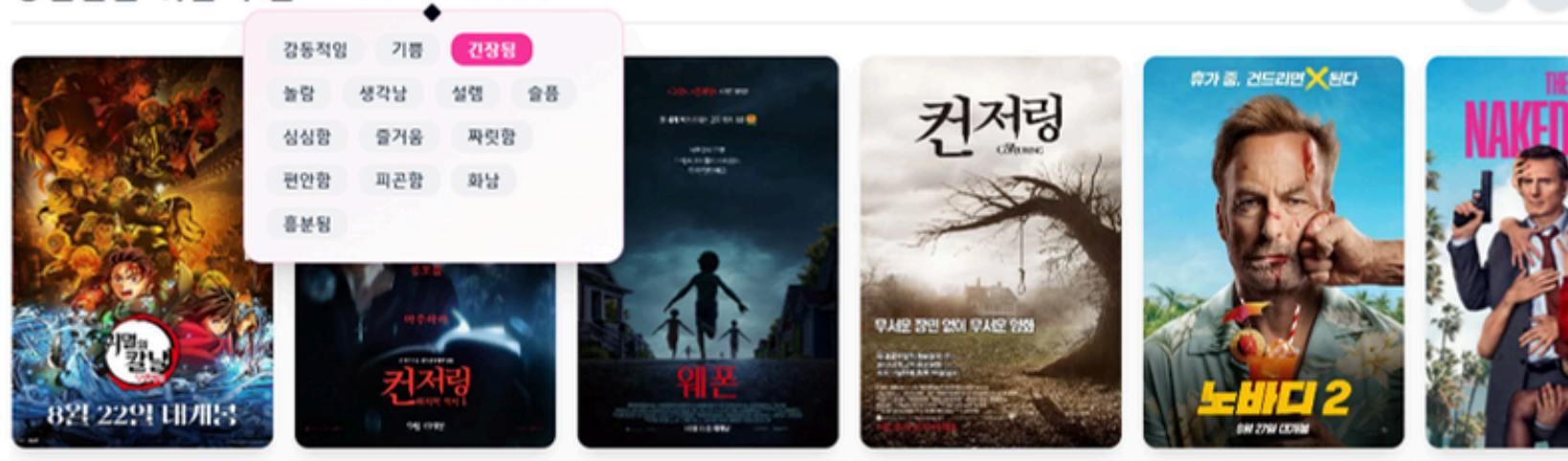
인기 영화 조회

```
/** 모든 영화 */
@GetMapping("/all")
public ResponseEntity<List<MovieInfoDto>> getAllMovies() {
    return ResponseEntity.ok(moviesService.getAllMovies());
}
```

단일 영화 조회

DATA ANALYSIS

당신만을 위한 추천 손에 땀을 쥐게 하는 순간 😓



장르에 따른 감정정보 태깅

```
private static final Map<String, String[]> genreToFeelingMap = Map.ofEntries(
    entry(k:"액션", new String[] { "화남", "긴장됨", "놀람", "짜릿함", "흥분됨" }),
    entry(k:"어드벤처", new String[] { "심심함", "설렘", "긴장됨", "흥분됨", "즐거움" }),
    entry(k:"애니메이션", new String[] { "기쁨", "편안함", "즐거움", "설렘", "심심함" }),
    entry(k:"코미디", new String[] { "기쁨", "편안함", "심심함", "즐거움", "설렘" }),
    entry(k:"범죄", new String[] { "화남", "긴장됨", "심심함", "놀람", "짜릿함" }),
    entry(k:"다큐멘터리", new String[] { "슬픔", "피곤함", "감동적임", "편안함", "생각남" }),
    entry(k:"드라마", new String[] { "슬픔", "기쁨", "편안함", "감동적임", "설렘" }),
    entry(k:"가족", new String[] { "기쁨", "편안함", "감동적임", "즐거움", "설렘" }),
    entry(k:"판타지", new String[] { "슬픔", "편안함", "흥분됨", "설렘", "놀람" }),
    entry(k:"역사", new String[] { "피곤함", "감동적임", "슬픔", "생각남", "설렘" }),
    entry(k:"공포", new String[] { "긴장됨", "놀람", "짜릿함", "화남", "심심함" }),
    entry(k:"음악", new String[] { "기쁨", "설렘", "편안함", "즐거움", "감동적임" }),
    entry(k:"미스터리", new String[] { "긴장됨", "놀람", "심심함", "짜릿함", "화남" }),
    entry(k:"로맨스", new String[] { "기쁨", "설렘", "슬픔", "편안함", "감동적임" }),
    entry(k:"SF", new String[] { "심심함", "흥분됨", "놀람", "설렘", "짜릿함" }),
    entry(k:"TV 영화", new String[] { "편안함", "심심함", "즐거움", "설렘", "감동적임" }),
    entry(k:"스릴러", new String[] { "긴장됨", "놀람", "화남", "짜릿함", "심심함" }),
    entry(k:"전쟁", new String[] { "화남", "긴장됨", "짜릿함", "슬픔", "감동적임" }),
    entry(k:"서부극", new String[] { "흥분됨", "짜릿함", "화남", "놀람", "심심함" }));
```

감정 태그

당신만을 위한 추천 진한 감동이 필요할 때 😓

두근두근 설레고 싶을 때 ❤️

즐겁고 행복한 순간 😊

박진감 넘치는 액션과 모험이 필요할 때 😡

심심할 때 가볍게 즐길 이야기 😊

```
// ♦ 감정 멘트
export const FEELING_MENTIONS: Record<string, string> = {
    슬픔:
    기쁨: "즐겁고 행복한 순간 😊",
    즐거움:
    흥분됨:
    짜릿함:
    설렘: "두근두근 설레고 싶을 때 ❤️",
    화남: "박진감 넘치는 액션과 모험이 필요할 때 😡",
    심심함: "심심할 때 가볍게 즐길 이야기 😊",
    놀람:
    피곤함:
    편안함:
    감동적임: "진한 감동이 필요할 때 😓",
    긴장됨:
    생각남:
};
```

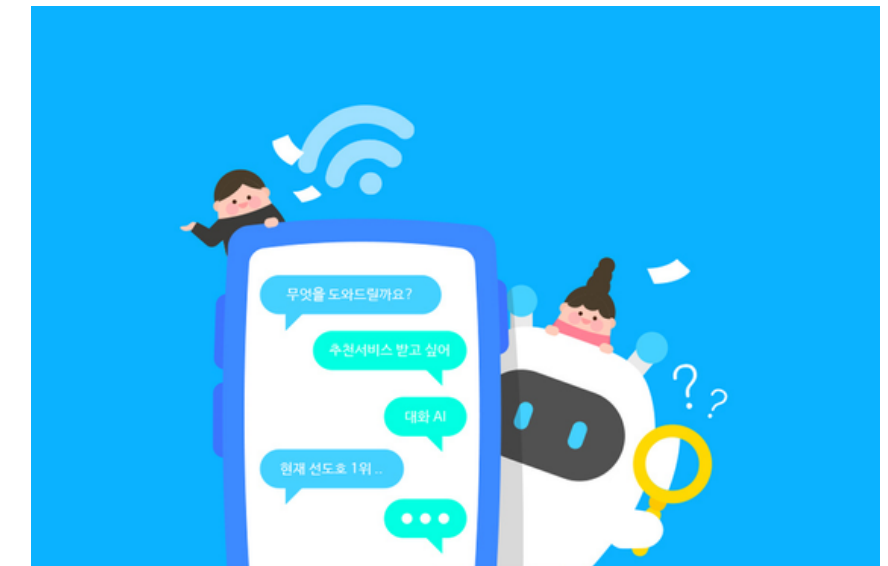
FUTURE PROSPECTS



SNS로 확장



수집요소 추가



챗봇 고도화

LESSONS LEARNED

김민재 : 처음 해보는 팀프로젝트 부족한 팀장을 지지해주는 팀원분들에게 너무 감사했습니다.

앞으로 어떤 방식으로 작업, 협업을 해야될지 알아가는 기회라고 생각합니다.

곽수영 : 시간이 없어서 아쉬웠다.
완성도를 더 높이고 싶었다.

박진우 : 프론트엔드 제작을 담당했었는데 실력이 부족하여 맡은 일을 제대로 해내지 못했습니다. 조원들에게 미안한 생각이 많이 듭니다.

임은상 : 이것저것 구상 많이 했지만 시간이 아쉽네요.
다들 최선을 다해주셔서 감사했습니다.
최종프로젝트에서 또 만나요.

임정수 : 실무하는 기분이었습니다.
앞으로도 더욱 정진하겠습니다.

오승현 : 실무적인 백엔드 서버 개발 과정을 체험할 수 있어서 유익한 시간이었다.

THANK YOU